

Running a PostgreSQL Cluster on Kubernetes Using the CloudNativePG Operator

End-to-End Architecture, Deployment, HA, Backup, Recovery, Monitoring, Security and
Operations

Document Type	Technical SOP / Reference Architecture
Technology	PostgreSQL + Kubernetes + CloudNativePG
Document Version	1.0
Publication Date	26 August 2026
Classification	Technical Reference / Training Use

SOURCE BASIS

Technical content in this SOP is based exclusively on the CloudNativePG official documentation referenced in the source article and was cross-checked against current CloudNativePG official documentation on 26 August 2026.

Document Control

Control Item	Value
Document Title	Running a PostgreSQL Cluster on Kubernetes Using the CloudNativePG Operator
Document ID	CNPG-PGSQL-K8S-SOP-001
Version	1.0
Status	Released for Technical Reference
Date	26 August 2026
Scope	CloudNativePG operator fundamentals through production-oriented PostgreSQL operations on Kubernetes
Primary Audience	PostgreSQL DBAs, Platform Engineers, SREs, Kubernetes Administrators, Technical Architects
Source Standard	CloudNativePG official documentation only

Revision History

Version	Date	Change Description
1.0	26-Aug-2026	Initial polished SOP derived from the supplied CloudNativePG technical article; current official documentation corrections incorporated.

Source and Accuracy Statement

Official-source statement

The technical guidance in this document is derived exclusively from the CloudNativePG official documentation referenced in the original notes and from current pages on cloudnative-pg.io used for verification. No third-party blog, vendor article, forum, or community post is used as a technical authority in this SOP.

Accuracy note: The material was cross-checked against current CloudNativePG official documentation as of 26 August 2026. A literal “100% accuracy” guarantee would be misleading because software documentation and supported behavior can change between releases. For production implementation, validate every manifest field and operating procedure against the documentation for the exact CloudNativePG and PostgreSQL versions deployed.

Document Conventions

Convention	Meaning
Command blocks	Commands executed with <code>kubectl</code> , <code>psql</code> , or a shell.
YAML blocks	Kubernetes / CloudNativePG declarative configuration examples.
SQL blocks	PostgreSQL verification and administration statements.
Production note	A control or decision that must be validated before production use.
Expected result	Illustrative output or behavior; exact values may vary by release and environment.

Table of Contents

This is a Microsoft Word TOC field based on Heading 1-3 styles. In Word, use Ctrl+A then F9 if page numbers need refresh after editing.

Document Control	2
Revision History	2
Source and Accuracy Statement	2
Document Conventions	2
1. Purpose, Scope and Audience.....	6
2. Background: From Patroni and etcd to CloudNativePG	6
3. What CloudNativePG Is	6
4. Why Use CloudNativePG	7
5. Kubernetes Operator Fundamentals	7
5.1 Custom Resources	7
5.2 Desired State and Reconciliation	7
6. CloudNativePG Architecture	8
6.1 Operator	8
6.2 Cluster CRD	8
6.3 Instance Manager	8
6.4 Persistent Storage.....	8
7. Prerequisites	8
8. Production Pre-Installation Checklist	9
9. Install the CloudNativePG Operator.....	9
10. Create a PostgreSQL Namespace	9
11. Create the First PostgreSQL Cluster.....	9
12. What the Operator Creates and Manages	10
13. Verify Cluster Status	10
14. PostgreSQL Services.....	10
15. Connect to PostgreSQL.....	11
16. Verify Streaming Replication	11
16.1 Primary-side check	11
16.2 Replica-side check.....	11
17. Replication Validation Lab	11
18. High Availability and Automatic Failover	12
19. Failover Test Procedure	12
20. Failover Is Not Zero Downtime.....	12
21. Switchover vs Failover	13
22. PostgreSQL Storage Design.....	13

23. Separate WAL Storage	13
24. PostgreSQL Configuration	14
25. Kubernetes Resource Requests and Limits	14
26. Secrets and Database Credentials.....	14
27. PostgreSQL Authentication and Roles.....	14
28. Application Connectivity	15
29. Read Scaling	15
30. Scaling PostgreSQL Instances	15
31. Backup Is Mandatory	15
32. CloudNativePG Backup Architecture	15
33. WAL Archiving and Point-in-Time Recovery	16
34. Recovery Model.....	16
35. Backup Policy Design	16
36. Monitoring Architecture	17
37. DBA Monitoring Baseline.....	17
37.1 PostgreSQL.....	17
37.2 Kubernetes	17
37.3 CloudNativePG	17
38. Kubernetes Node Failure Scenario.....	18
39. Failure Scenarios to Test	18
40. Scheduling and Failure Domains	18
41. Production Reference Architecture	19
42. Connection Pooling	19
43. PostgreSQL Minor Upgrades.....	19
44. CloudNativePG Operator Upgrades	19
45. PostgreSQL Major Upgrades.....	20
46. Cluster Bootstrap Methods	20
47. Migration from Patroni/VM PostgreSQL	20
48. Disaster Recovery	20
49. HA vs DR.....	21
50. Security Checklist	21
51. Production Configuration Principles	21
52. Production-Oriented Manifest Skeleton	22
53. Image Version Control	22
54. Day-to-Day DBA Command Reference	23
55. Troubleshooting Flow	23
56. Scenario: PVC Nearly Full	23
57. Scenario: Replica Lag	24

58. Scenario: Accidental DELETE	24
59. Scenario: Primary Pod Deleted	24
60. Scenario: Kubernetes Node Maintenance	24
61. Production Failure-Test Matrix.....	25
62. Mapping Patroni Knowledge to CloudNativePG.....	25
63. Traditional DBA vs Kubernetes Operating Model	25
64. What CloudNativePG Automates.....	26
65. What CloudNativePG Does Not Eliminate	26
66. Recommended Hands-On Lab	26
67. Final Architecture Summary	27
68. Key Takeaways	27
69. Official CloudNativePG References	28
Appendix A. Quick Validation Checklist	28
Appendix B. Change-Control Reminder	28

1. Purpose, Scope and Audience

This SOP provides an end-to-end technical foundation for running a highly available PostgreSQL cluster on Kubernetes using the CloudNativePG operator. It is designed for engineers who understand PostgreSQL and want to move from manually assembled HA stacks, such as Patroni plus etcd, to a Kubernetes-native operator model.

- Covers core concepts, architecture and operator behavior.
- Builds a three-instance PostgreSQL cluster for hands-on practice.
- Explains services, replication, failover, storage, backup/PITR, monitoring, security, upgrades and DR.
- Adds production readiness and troubleshooting checkpoints.

2. Background: From Patroni and etcd to CloudNativePG

A traditional PostgreSQL HA design commonly combines several independently managed components:

```
PostgreSQL
+
Patroni
+
etcd / Consul
+
HAProxy / Load Balancer
+
Backup tooling
+
Monitoring
```

This model is valid and widely used, but Kubernetes already provides an orchestration and control-plane layer. CloudNativePG extends Kubernetes with PostgreSQL-specific lifecycle management so that the cluster is managed declaratively rather than assembled one component at a time.

Key idea

CloudNativePG is not Patroni running inside Kubernetes. It has its own operator and instance-manager architecture and uses Kubernetes-native resources for coordination and service management.

3. What CloudNativePG Is

CloudNativePG is an open-source Kubernetes operator for PostgreSQL. Its central custom resource is the CloudNativePG Cluster resource, which represents an entire PostgreSQL cluster rather than a single server.

```
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-prod
spec:
  instances: 3
```

When instances is greater than one, CloudNativePG manages one primary and the remaining instances as replicas, including automated failover and rolling-update workflows.

4. Why Use CloudNativePG

Manual HA Responsibility	CloudNativePG / Kubernetes Equivalent
PostgreSQL process and instance lifecycle	CloudNativePG operator + instance manager
DCS such as etcd/Consul	Kubernetes-native coordination, including Lease-based promotion control
HAProxy endpoint	Kubernetes Services and, when needed, external load balancing
systemd-managed servers	Kubernetes Pods
Server disk/LVM	PVC + StorageClass + CSI-backed persistent storage
Patroni YAML and provisioning scripts	Declarative Cluster resource and related CRDs
Manual replica lifecycle	Operator-managed replication and replica reconciliation

The operator reduces infrastructure lifecycle work, but it does not remove the need for PostgreSQL engineering. DBAs still own query performance, data integrity, capacity, vacuum, backup, recovery, security and RPO/RTO decisions.

5. Kubernetes Operator Fundamentals

5.1 Custom Resources

A Kubernetes operator extends the Kubernetes API with application-specific resources. CloudNativePG exposes resources for PostgreSQL lifecycle management, including Cluster, Backup, ScheduledBackup, Pooler, and other supported CRDs depending on release.

5.2 Desired State and Reconciliation

```
Desired state: instances = 3
Actual state:  instances = 2
```

```
CloudNativePG detects the difference
```

```
↓
```

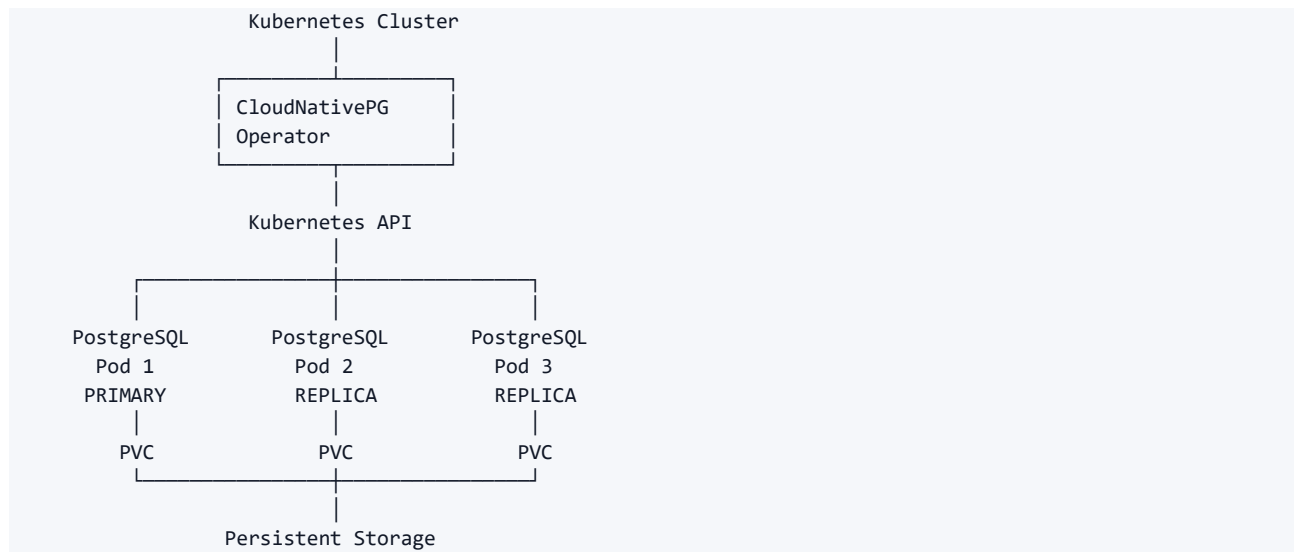
```
Reconciliation starts
```

```
↓
```

```
Cluster is returned toward the desired state
```

This reconciliation loop is the core operating model. A DBA declares the target state; Kubernetes and the operator continuously work to maintain it.

6. CloudNativePG Architecture



6.1 Operator

The operator watches CloudNativePG resources and reconciles Kubernetes and PostgreSQL state. Manifest-based installations commonly deploy the controller into the cnpg-system namespace.

6.2 Cluster CRD

The Cluster custom resource describes the PostgreSQL topology and many database, storage, networking and lifecycle settings.

6.3 Instance Manager

Each PostgreSQL pod contains the CloudNativePG instance manager, which participates in PostgreSQL process management and the cluster lifecycle coordinated by the operator.

6.4 Persistent Storage

PostgreSQL data must live on persistent storage. Pods are replaceable compute objects; database data must survive pod replacement through PVC-backed storage.

7. Prerequisites

- ☐ Supported Kubernetes cluster is available and healthy.
- ☐ kubectl is installed and configured.
- ☐ Worker nodes have sufficient CPU and memory.
- ☐ A functional CSI driver and StorageClass are available.
- ☐ Persistent volume provisioning and expansion have been validated.
- ☐ Cluster DNS and required network paths are working.
- ☐ Container registry access is available.
- ☐ PostgreSQL image/version has been selected and pinned.

- ☐ Backup target and credential strategy are defined.
- ☐ Monitoring, RPO/RTO and DR requirements are documented.

```
kubectl cluster-info
kubectl get nodes
kubectl get storageclass
```

8. Production Pre-Installation Checklist

- ☐ Kubernetes control plane and worker nodes are healthy.
- ☐ Production StorageClass has been benchmarked for latency, IOPS and throughput.
- ☐ Storage failure and expansion behavior is understood.
- ☐ Failure-domain and zone placement is designed.
- ☐ CPU/memory sizing is documented.
- ☐ TLS and pg_hba strategy is defined.
- ☐ Namespace and Kubernetes RBAC boundaries are defined.
- ☐ Database roles and secrets-management approach are defined.
- ☐ Backup, WAL archive, retention and restore-test policy are approved.
- ☐ Monitoring and alerting integration is ready.
- ☐ Maintenance, upgrade and DR runbooks are defined.

9. Install the CloudNativePG Operator

Use the official installation instructions for the exact release selected. For manifest-based installation, the pattern is:

```
kubectl apply --server-side -f <OFFICIAL-CNPG-RELEASE-MANIFEST>
```

Verify the controller and CRDs:

```
kubectl get pods -n cnpg-system
kubectl get deployment -n cnpg-system
kubectl get crd | grep cnpg
```

Production control

Never copy an old release URL from a training document into production. Pin the exact supported CloudNativePG release and review its installation and upgrade notes.

10. Create a PostgreSQL Namespace

```
kubectl create namespace postgres
kubectl get namespace postgres
```

Using a dedicated namespace makes RBAC, quotas, network policy, secrets and lifecycle ownership easier to control.

11. Create the First PostgreSQL Cluster

Create pg-cluster.yaml:

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-cluster
  namespace: postgres
spec:
  instances: 3
  storage:
    size: 10Gi

kubectl apply -f pg-cluster.yaml
kubectl get cluster -n postgres
kubectl get pods -n postgres

```

With instances: 3, the desired topology is one primary and two replicas. Pod numbers must not be used to assume roles; always verify the current role.

12. What the Operator Creates and Manages

A small Cluster manifest results in a complete database service. Depending on the configuration and release, CloudNativePG manages Kubernetes and PostgreSQL resources such as pods, services, secrets, PVCs, RBAC-related objects, PodDisruptionBudgets, replication settings and cluster lifecycle operations.

- The operator creates and maintains the requested number of PostgreSQL instances.
- It manages replication and HA behavior.
- It reconciles failed or missing standby instances.
- It coordinates rolling update and primary-transition operations.

13. Verify Cluster Status

```

kubectl get cluster -n postgres
kubectl describe cluster pg-cluster -n postgres
kubectl get pods -n postgres -o wide

```

The -o wide output is useful for checking which Kubernetes nodes currently host the database pods and whether HA placement matches the intended failure-domain design.

14. PostgreSQL Services

Service Pattern	Purpose
pg-cluster-rw	Primary endpoint for read/write connections.
pg-cluster-ro	Read-only endpoint targeting replicas.
pg-cluster-r	Generic read service that can target PostgreSQL instances according to the operator service model.

```

kubectl get svc -n postgres

```

Applications should use stable services instead of individual pod names. When the primary changes, CloudNativePG adjusts the primary service routing to the current primary.

15. Connect to PostgreSQL

```
kubectl get pods -n postgres
kubectl exec -it -n postgres <pod-name> -- psql

SELECT version();
SELECT pg_is_in_recovery();
```

pg_is_in_recovery()	Meaning
false	Current instance is primary.
true	Current instance is in recovery and operating as a replica.

16. Verify Streaming Replication

16.1 Primary-side check

```
SELECT application_name, client_addr, state, sync_state
FROM pg_stat_replication;
```

16.2 Replica-side check

```
SELECT pg_is_in_recovery();

SELECT status, sender_host, latest_end_lsn
FROM pg_stat_wal_receiver;
```

A healthy HA cluster should show the expected standby connections and streaming state. Exact application names and synchronous/asynchronous behavior depend on configuration.

17. Replication Validation Lab

```
CREATE DATABASE cnpg_lab;
\c cnpg_lab

CREATE TABLE employee (
    emp_id      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    emp_name    varchar(100),
    department  varchar(50),
    created_at  timestamptz DEFAULT now()
);

INSERT INTO employee(emp_name, department)
VALUES
('Praveen', 'Database Engineering'),
('Raj', 'Cloud Engineering'),
('John', 'Application Engineering');

SELECT * FROM employee;
```

Verify the rows from a replica. This confirms that PostgreSQL physical streaming replication is carrying the changes to the standby.

18. High Availability and Automatic Failover

CloudNativePG provides automated failover when it detects primary failure. The operator selects an eligible, sufficiently aligned replica as target primary and coordinates promotion. Current CloudNativePG documentation describes a per-cluster Kubernetes Lease used as a mutex so that at most one instance promotes as primary.

Before failure

```
Primary A → Replica B
           |
           → Replica C
```

After failover

```
Primary A   FAILED
Replica B   PROMOTED TO PRIMARY
Replica C   FOLLOWS NEW PRIMARY
```

Important

Automatic failover improves availability; it does not imply zero downtime. Applications must tolerate the connection break and reconnect to the stable service.

19. Failover Test Procedure

1. Identify the current primary from CloudNativePG status and/or `pg_is_in_recovery()`.
2. Confirm both replicas are healthy and replication is current enough for the planned test.
3. Delete the current primary pod.
4. Watch cluster and pod state during promotion.
5. Verify that the `-rw` service resolves to the new primary.
6. Verify application connectivity and writes.
7. Verify the failed instance is rebuilt or returned as a standby and cluster health returns to desired state.

```
kubectl get cluster pg-cluster -n postgres
kubectl delete pod <current-primary-pod> -n postgres
kubectl get pods -n postgres -w
kubectl get cluster pg-cluster -n postgres -w
```

20. Failover Is Not Zero Downtime

```
Primary unavailable
↓
Failure detected
↓
Promotion coordinated
↓
New primary ready
↓
Service routing converges
↓
Application reconnects
```

- Set realistic connection timeouts.
- Use retry logic appropriate to the application transaction model.

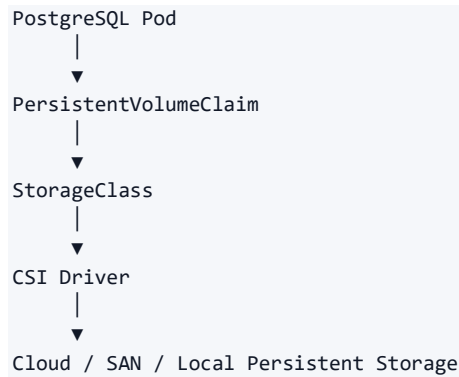
- Use connection pooling when useful, but validate pool behavior during failover.
- Measure real RTO and RPO through failure testing instead of assuming them.

21. Switchover vs Failover

Operation	Use Case	Nature
Switchover	Planned maintenance, controlled primary move, some rolling updates.	Controlled / planned
Failover	Primary or infrastructure failure.	Unplanned / automatic recovery

22. PostgreSQL Storage Design

Storage is a primary production design decision. Size alone is not enough. Evaluate IOPS, latency, throughput, durability, topology, expansion, snapshot behavior and failure recovery.



Production control

Benchmark the selected storage with a workload representative of PostgreSQL behavior before production. Validate behavior during node, zone and storage disruptions.

23. Separate WAL Storage

CloudNativePG supports dedicated WAL storage when the workload and storage architecture justify it.

```

spec:
  storage:
    size: 500Gi
    storageClass: postgres-data

  walStorage:
    size: 100Gi
    storageClass: postgres-wal
  
```

WAL capacity must be based on measured WAL generation, archive behavior, replication lag, checkpoints and recovery requirements. Separate WAL storage is an architectural option, not a universal rule.

24. PostgreSQL Configuration

```
spec:
  postgresql:
    parameters:
      max_connections: "300"
      shared_buffers: "2GB"
      effective_cache_size: "6GB"
      work_mem: "8MB"
      maintenance_work_mem: "512MB"
```

Sizing warning

These values are examples only. PostgreSQL parameters must be sized against pod CPU/memory, workload, concurrency, pooling, maintenance operations and storage performance.

25. Kubernetes Resource Requests and Limits

```
resources:
  requests:
    memory: "8Gi"
    cpu: "2"
  limits:
    memory: "8Gi"
    cpu: "4"
```

PostgreSQL memory sizing must fit inside the container boundary. Do not tune PostgreSQL as though the database owns the entire Kubernetes node.

26. Secrets and Database Credentials

Production passwords must not be stored as plain text in Git-managed manifests. Use Kubernetes Secrets and, where required, an approved enterprise secrets-management solution. Treat Kubernetes Secrets as a mechanism that still requires RBAC, encryption, rotation and audit controls.

```
kubectl get secrets -n postgres
kubectl describe secret <secret-name> -n postgres
```

27. PostgreSQL Authentication and Roles

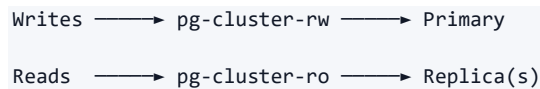
- Use dedicated application roles instead of the postgres superuser.
- Create separate DBA, application, reporting and monitoring identities as required.
- Apply least privilege at database, schema, table and function levels.
- Define pg_hba rules deliberately and combine them with TLS and Kubernetes network controls.
- Implement password/secret rotation and audit procedures.

28. Application Connectivity



Stable Kubernetes Services decouple applications from the physical identity of a PostgreSQL pod.

29. Read Scaling



Read-only services can be useful for reporting, BI, dashboards and read-only APIs. Applications must tolerate replica lag and must not assume read-after-write consistency from a standby unless the design explicitly guarantees it.

30. Scaling PostgreSQL Instances

```
spec:
  instances: 5
```

Changing instances from 3 to 5 asks CloudNativePG to reconcile toward one primary and four replicas. More replicas can improve HA/read capacity and operational flexibility, but they do not increase the write capacity of a single-primary PostgreSQL architecture.

31. Backup Is Mandatory

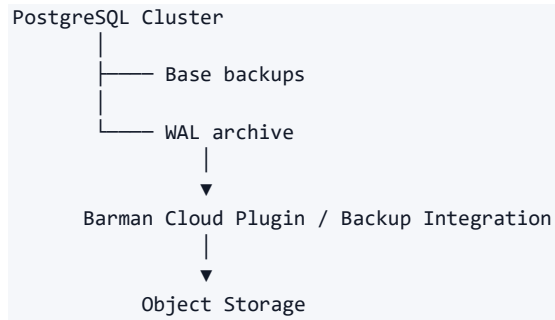
Core principle

Replication is not backup. High availability is not backup. A replica is not a backup.

Logical mistakes such as DROP TABLE or DELETE are replicated. A recoverability strategy therefore requires independent backups and, when point-in-time recovery is required, WAL archiving.

32. CloudNativePG Backup Architecture

Current CloudNativePG supports physical backup workflows through CNPG-I plugins and Kubernetes volume snapshots. The official Barman Cloud Plugin is the supported object-storage integration for continuous physical backups and WAL archiving in the newer plugin architecture.



Supported object-store families documented by CloudNativePG include S3/S3-compatible storage, Azure Blob Storage and Google Cloud Storage, depending on the integration used.

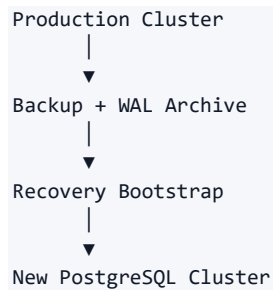
33. WAL Archiving and Point-in-Time Recovery

Base Backup + Archived WAL = Point-in-Time Recovery capability

Example: if a destructive change occurs at 11:43, the recovery target may be a point immediately before that event, provided the backup and required WAL archive are available.

34. Recovery Model

CloudNativePG recovery normally bootstraps a new PostgreSQL cluster from an existing backup and replays available WAL to the requested target. This cloud-native model preserves the original cluster for investigation while the recovered cluster is validated.



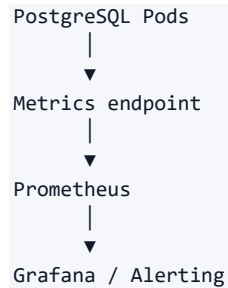
35. Backup Policy Design

Control	Define Explicitly
RPO	Maximum acceptable data loss.
RTO	Maximum acceptable recovery duration.
Base backup	Frequency and scheduling.
WAL archive	Continuous archive target and health checks.
Retention	How long backups/WAL are retained.
Security	Encryption, identity and least-privilege access.
Restore testing	Frequency, ownership and success criteria.
Alerting	Backup/WAL failures and storage-capacity alerts.

Operational rule

A backup that has never been restored is an unproven backup.

36. Monitoring Architecture



CloudNativePG exposes PostgreSQL/operator metrics suitable for Prometheus-based monitoring, and the official documentation describes integration patterns such as PodMonitor when Prometheus Operator is used.

37. DBA Monitoring Baseline

37.1 PostgreSQL

- Connections, transaction rate, long-running queries, locks and deadlocks.
- Checkpoints, WAL generation, WAL archiving and replication lag.
- Database/table/index growth and temporary-file activity.
- Autovacuum effectiveness, vacuum progress and transaction-ID age.
- Query performance, cache behavior and maintenance operations.

37.2 Kubernetes

- Pod status and restart count, CPU and memory usage.
- Node health/pressure and scheduling events.
- PVC utilization, storage latency and capacity growth.

37.3 CloudNativePG

- Cluster/primary status, replica health and failover/switchover events.
- Backup/WAL-archive state and reconciliation errors.

38. Kubernetes Node Failure Scenario

```

Worker node fails
↓
Primary becomes unavailable
↓
CloudNativePG detects unhealthy primary
↓
Eligible replica promoted
↓
-rw service follows new primary
↓
Cluster reconciles toward desired state

```

Test this scenario in a controlled non-production environment. Pod deletion alone does not exercise all node, networking and storage behaviors.

39. Failure Scenarios to Test

Scenario	What It Validates
Primary pod failure	Promotion and application reconnection.
Replica pod failure	Standby recreation and desired-state reconciliation.
Worker-node failure	Scheduling, promotion and storage behavior.
Worker-node drain	Maintenance safety and disruption controls.
Network disruption	Failure detection and split-brain protections.
Storage pressure/failure	Database response, alerts and recovery procedures.
Backup target failure	WAL/archive alerts and RPO risk visibility.

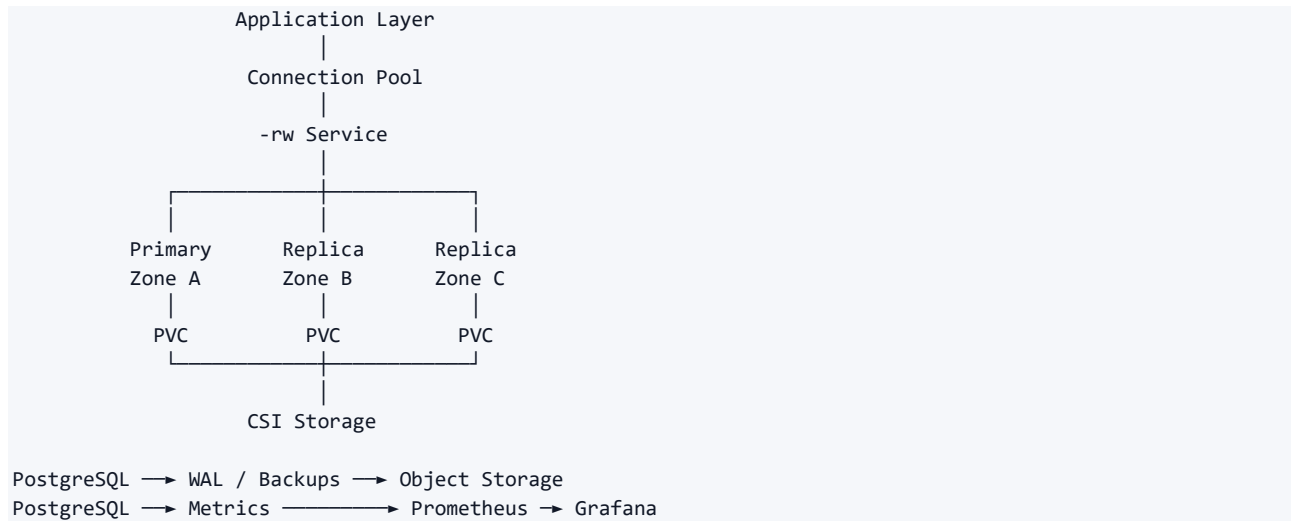
40. Scheduling and Failure Domains

Bad placement:
worker01: Primary + Replica 1 + Replica 2

Better HA placement:
worker01: Primary
worker02: Replica 1
worker03: Replica 2

Use appropriate pod anti-affinity, node affinity, topology spread constraints and storage topology so that a single node or failure domain does not remove the entire database service.

41. Production Reference Architecture



42. Connection Pooling

Kubernetes application scaling can create large numbers of database connections. CloudNativePG provides a Pooler resource for PgBouncer-based connection pooling.

$100 \text{ application pods} \times 20 \text{ connections} = 2,000 \text{ PostgreSQL connections}$

Pool sizing, transaction/session mode and application compatibility must be tested; pooling is not a substitute for workload-specific connection management.

43. PostgreSQL Minor Upgrades

CloudNativePG supports declarative workload updates using the PostgreSQL image reference. In HA clusters, rolling updates normally update replicas first. The primary update behavior depends on `primaryUpdateStrategy` and can involve a controlled switchover.

- Pin and test the target image.
- Validate PostgreSQL extensions and application compatibility.
- Verify backups and recovery before the change.
- Observe cluster status and replication during the rollout.

44. CloudNativePG Operator Upgrades

Operator upgrades require release-note and compatibility review. Current documentation describes both rolling workload-update behavior and newer in-place instance-manager update capabilities. Exact impact depends on release and configuration, so the change plan must be based on the version-specific upgrade guide.

45. PostgreSQL Major Upgrades

Current documentation correction

Current CloudNativePG releases support declarative offline in-place PostgreSQL major upgrades using `pg_upgrade`. During this process the database cluster is taken offline, upgrade conditions are validated, `pg_upgrade` is run, and replicas are recreated afterward. This behavior must be validated against the exact CloudNativePG release in use.

Major upgrades remain a separate change project because they require extension compatibility checks, backups, rollback planning, outage planning, performance testing and post-upgrade validation.

46. Cluster Bootstrap Methods

Method	Purpose
initdb	Create a new PostgreSQL cluster.
pg_basebackup	Bootstrap/clone from a compatible existing PostgreSQL source where supported.
recovery	Create a cluster from an accessible backup and WAL archive / recovery source.

47. Migration from Patroni/VM PostgreSQL

Current: VM PostgreSQL + Patroni + etcd

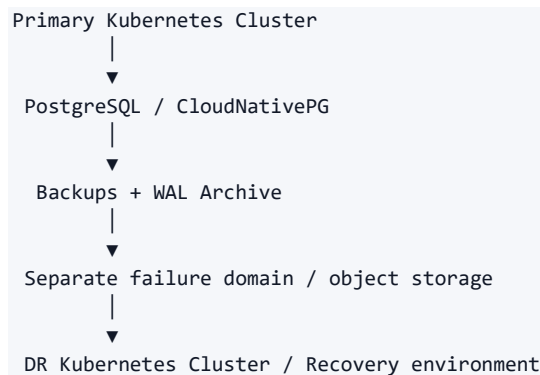


Target: Kubernetes + CloudNativePG + PostgreSQL

Choose a migration method based on database size, PostgreSQL versions, extensions, downtime tolerance, network capacity, RPO/RTO and rollback requirements. Possible techniques include logical dump/restore, logical replication, base-backup-based bootstrap, or backup/recovery workflows as supported by the target design.

48. Disaster Recovery

Three PostgreSQL pods in one Kubernetes cluster provide HA but do not by themselves provide disaster recovery from loss of the Kubernetes cluster, region or major shared infrastructure.



CloudNativePG also documents replica-cluster/distributed-topology patterns that can be used in DR-oriented designs.

49. HA vs DR

Capability	Typically Protects Against	Design Scope
High Availability	Pod, PostgreSQL process, node and selected infrastructure failures.	Usually within a Kubernetes cluster/failure-domain design.
Disaster Recovery	Cluster, site/region, catastrophic storage or major operational failure.	Separate failure domain, backup location and/or Kubernetes cluster.

50. Security Checklist

- ☐ TLS is implemented for required database connections.
- ☐ Application roles use least privilege and do not use postgres superuser.
- ☐ Kubernetes RBAC and namespace access are restricted.
- ☐ NetworkPolicy is applied where appropriate.
- ☐ Secrets and backup credentials are protected and rotated.
- ☐ Backup object-store identity is least privilege.
- ☐ PostgreSQL/container images are trusted and pinned.
- ☐ Extensions are reviewed and controlled.
- ☐ Audit and access-review requirements are documented.
- ☐ Certificate and secret rotation procedures are tested.

51. Production Configuration Principles

A production Cluster manifest normally needs decisions beyond instances and storage size. Treat the following as an architecture checklist rather than a copy-and-run template:

- PostgreSQL image/version and extension strategy.
- Instance count, CPU/memory and storage classes.
- Data and optional WAL storage sizing.
- PostgreSQL parameters and connection limits.
- TLS, pg_hba, users/roles and secret lifecycle.
- Scheduling, topology and disruption controls.
- Backup plugin, WAL archiving, retention and restore testing.
- Monitoring, alerting and operator update strategy.
- DR architecture and recovery ownership.

52. Production-Oriented Manifest Skeleton

```

apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata:
  name: pg-prod
  namespace: postgres
spec:
  instances: 3
  imageName: <PINNED-POSTGRESQL-IMAGE>

  resources:
    requests:
      memory: "8Gi"
      cpu: "2"
    limits:
      memory: "8Gi"
      cpu: "4"

  storage:
    storageClass: postgres-data
    size: 500Gi

  walStorage:
    storageClass: postgres-wal
    size: 100Gi

  postgresql:
    parameters:
      max_connections: "300"

  bootstrap:
    initdb:
      database: appdb
      owner: appuser

```

Not a production drop-in

Validate every field against the API reference for the exact CloudNativePG release and add the environment-specific TLS, authentication, backup, monitoring, topology, update and security controls.

53. Image Version Control

Use deterministic, tested image versions. Avoid floating tags such as latest for production. Promote images through development, QA/staging, failover/recovery testing and production change control.

```

DEV → QA → STAGE → PROD
  compatibility
  failover
  backup/restore
  performance
  security validation

```

54. Day-to-Day DBA Command Reference

Purpose	Command
Cluster status	<code>kubectl get cluster -n postgres</code>
Pod status	<code>kubectl get pods -n postgres</code>
Pod/node placement	<code>kubectl get pods -n postgres -o wide</code>
Services	<code>kubectl get svc -n postgres</code>
PVCs	<code>kubectl get pvc -n postgres</code>
Detailed cluster	<code>kubectl describe cluster pg-cluster -n postgres</code>
Pod logs	<code>kubectl logs <pod-name> -n postgres</code>
psql session	<code>kubectl exec -it <pod-name> -n postgres -- psql</code>
Role check	<code>SELECT pg_is_in_recovery();</code>
Primary replication	<code>SELECT * FROM pg_stat_replication;</code>
Replica receiver	<code>SELECT * FROM pg_stat_wal_receiver;</code>
Replication slots	<code>SELECT * FROM pg_replication_slots;</code>

55. Troubleshooting Flow

```

1. Cluster status
  ↓
2. Pods and current primary
  ↓
3. Services/endpoints
  ↓
4. PostgreSQL health
  ↓
5. Replication
  ↓
6. PVC/storage
  ↓
7. Kubernetes nodes
  ↓
8. Operator and events
  ↓
9. Application connectivity

kubectl get cluster -n postgres
kubectl get pods -n postgres -o wide
kubectl get svc -n postgres
kubectl get pvc -n postgres
kubectl get events -n postgres --sort-by=.lastTimestamp
kubectl get nodes

```

56. Scenario: PVC Nearly Full

Example: a 500 GB database PVC has reached 470 GB used. Do not wait for 100% utilization.

- Check table/index/database growth.
- Check retained WAL and replication slots.
- Check archive failures and replica lag.
- Check temporary files and unusual workload growth.
- Review storage expansion and capacity-management procedures.

57. Scenario: Replica Lag

```
SELECT application_name, state, sent_lsn, write_lsn, flush_lsn, replay_lsn
FROM pg_stat_replication;
```

Possible causes include network latency, storage latency, heavy WAL generation, CPU/I/O pressure and workload on the standby. Kubernetes does not remove PostgreSQL replication-performance constraints.

58. Scenario: Accidental DELETE

```
DELETE FROM customer;
  |
  ▼
  WAL
  |
  ▼
Replica(s)
```

Failover will not recover logically deleted data because the delete is valid replicated WAL. The recovery path is backup plus WAL archive plus point-in-time recovery, if configured and within retention.

59. Scenario: Primary Pod Deleted

1. Confirm failover started and identify the new primary.
2. Verify -rw service routing and application connectivity.
3. Verify both replication and the rebuilt former primary/standby.
4. Check cluster status for healthy desired state.
5. Check backup/WAL-archive health after the event.

60. Scenario: Kubernetes Node Maintenance

1. List PostgreSQL pods with node placement.
2. Identify the current primary and confirm replicas are healthy.
3. Review PodDisruptionBudget and maintenance constraints.
4. Decide whether a controlled switchover is required.
5. Drain/maintain the node according to platform procedure.
6. Monitor cluster, service and replication state.
7. Validate PostgreSQL and application health after maintenance.

61. Production Failure-Test Matrix

- ☐ Primary pod deletion tested.
- ☐ Replica pod deletion tested.
- ☐ Worker-node failure tested.
- ☐ Worker-node drain tested.
- ☐ Controlled switchover tested.
- ☐ Application reconnect after failover tested.
- ☐ Replica-lag alert tested.
- ☐ PVC expansion tested.
- ☐ Storage-pressure alert tested.
- ☐ Backup and backup-failure alert tested.
- ☐ Full restore tested.
- ☐ PITR tested.
- ☐ Operator restart/upgrade tested.
- ☐ PostgreSQL minor update tested.
- ☐ Secret/TLS rotation tested.
- ☐ Failure-domain loss tested.
- ☐ DR recovery tested.

Record actual RTO, actual RPO, application impact, operator behavior, recovery steps and escalation ownership for each test.

62. Mapping Patroni Knowledge to CloudNativePG

Existing PostgreSQL/Patroni Concept	CloudNativePG Relevance
Primary / replica	Same PostgreSQL roles.
Streaming replication / WAL	Same PostgreSQL fundamentals.
Failover / switchover	Operator-managed through Kubernetes-native control.
etcd DCS	Replaced by CloudNativePG/Kubernetes coordination model.
HAProxy stable endpoint	Kubernetes Services; Pooler/external LB as architecture requires.
Replication slots	Managed by CloudNativePG for HA replicas; additional options exist by release.
RPO / RTO / split brain	Still core HA design concerns.

63. Traditional DBA vs Kubernetes Operating Model

Traditional Server Model	Cloud-Native Model
--------------------------	--------------------

Traditional Server Model	Cloud-Native Model
“Repair pg01.”	“Restore the declared PostgreSQL service state.”
Server identity is central.	Service and desired state are central.
Provision server then configure DB.	Declare Cluster spec and reconcile.
Manual HA component integration.	Operator integrates supported PostgreSQL lifecycle functions with Kubernetes.

64. What CloudNativePG Automates

- PostgreSQL cluster creation and replica lifecycle.
- Streaming-replication orchestration and replication slots for HA replicas.
- Automated failover and controlled switchover workflows.
- Stable Kubernetes service routing.
- Persistent-storage attachment and replica recreation.
- Rolling updates and supported upgrade workflows.
- Backup/recovery orchestration through supported mechanisms.
- Metrics/monitoring integration and continuous reconciliation.

65. What CloudNativePG Does Not Eliminate

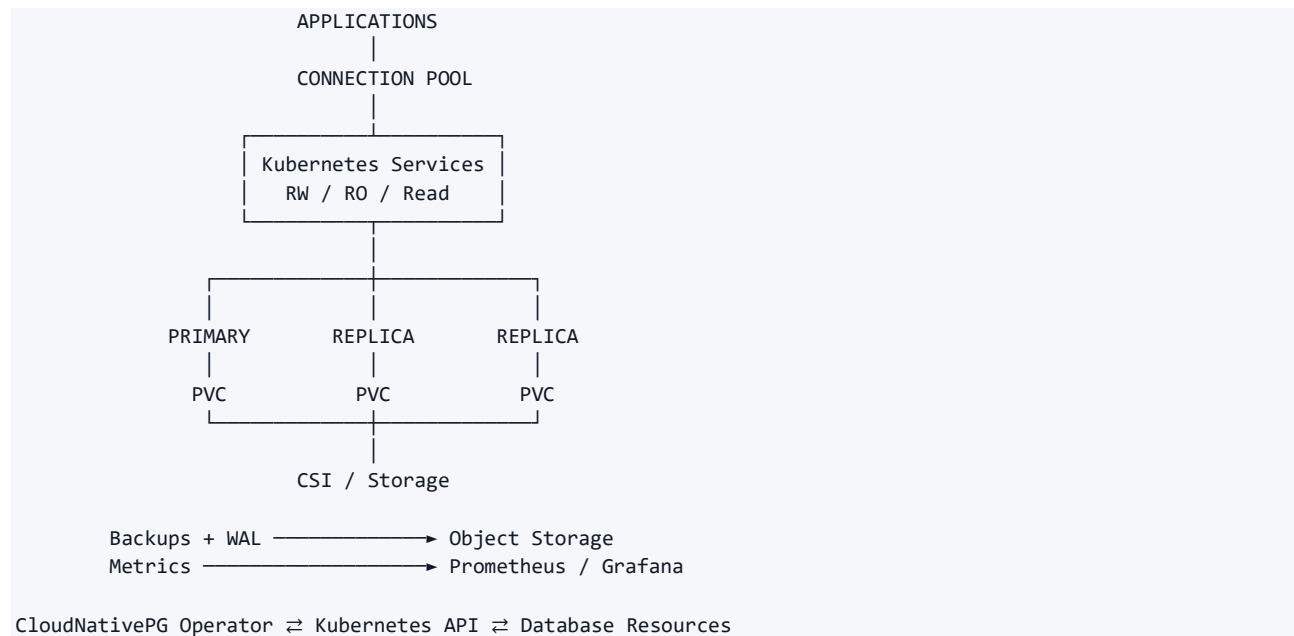
- PostgreSQL query tuning and indexing.
- Vacuum/autovacuum and bloat management.
- Locking/transaction troubleshooting.
- Storage and capacity planning.
- Security architecture and privilege design.
- Backup policy, restore testing and DR governance.
- RPO/RTO definition and validation.
- Application transaction/retry design.

66. Recommended Hands-On Lab

1. Create a three-instance cluster.
2. Connect through the read/write service.
3. Connect through the read-only service.
4. Create test data and verify streaming replication.
5. Delete a replica and observe recreation.
6. Delete the primary and observe automatic failover.
7. Identify the new primary and verify application connectivity.
8. Perform a controlled switchover.

9. Scale the cluster out and back in.
10. Configure supported physical backups and WAL archiving.
11. Perform a full restore and PITR.
12. Test node maintenance and failure.
13. Monitor replication lag and storage usage.
14. Perform a PostgreSQL minor update and operator-upgrade test.
15. Document measured RTO/RPO and operational findings.

67. Final Architecture Summary



68. Key Takeaways

1. CloudNativePG is a Kubernetes-native PostgreSQL operator, not a Patroni wrapper.
2. The Cluster resource defines the desired PostgreSQL topology and lifecycle configuration.
3. Stable Kubernetes services isolate applications from current pod/primary identity.
4. Automatic failover improves availability but does not mean zero downtime.
5. Persistent storage quality, topology and capacity are critical to PostgreSQL reliability.
6. Replication and HA do not replace backup, WAL archiving or restore testing.
7. HA inside one Kubernetes cluster is not the same as disaster recovery.
8. Monitoring must include PostgreSQL, Kubernetes and CloudNativePG/operator state.
9. Current releases include advanced lifecycle features such as declarative offline in-place major upgrades; always follow release-specific documentation.
10. Operator automation reduces operational toil but PostgreSQL expertise remains essential.

69. Official CloudNativePG References

Official Reference	URL
CloudNativePG Documentation	https://cloudnative-pg.io/documentation/
Operator Capability Levels	https://cloudnative-pg.io/documentation/current/operator_capability_levels/
Failover	https://cloudnative-pg.io/documentation/current/failover/
Backup	https://cloudnative-pg.io/documentation/current/backup/
Recovery	https://cloudnative-pg.io/documentation/current/recovery/
Monitoring	https://cloudnative-pg.io/documentation/current/monitoring/
PostgreSQL Upgrades	https://cloudnative-pg.io/documentation/current/postgres_upgrades/
Replica Clusters	https://cloudnative-pg.io/documentation/current/replica_cluster/
Bootstrap	https://cloudnative-pg.io/documentation/current/bootstrap/

Version-control note: “current” documentation changes over time. For a production change, use and retain the versioned documentation matching the deployed CloudNativePG release.

Appendix A. Quick Validation Checklist

- ☐ Operator is healthy.
- ☐ Cluster reports healthy status.
- ☐ Primary and replica roles are verified.
- ☐ Replication is streaming and lag is within target.
- ☐ -rw and -ro services behave as designed.
- ☐ PVCs have safe capacity headroom.
- ☐ Backup and WAL archive are successful.
- ☐ Restore/PITR have been tested.
- ☐ Monitoring and alerting are active.
- ☐ Node/failure-domain placement meets HA design.
- ☐ Application failover/reconnect behavior is validated.
- ☐ Production manifests match the exact CloudNativePG API version deployed.

Appendix B. Change-Control Reminder

Before production changes

Review the release-specific CloudNativePG documentation, PostgreSQL release notes, extension compatibility, backup/recovery readiness, rollback plan, maintenance window, application impact and monitoring coverage. Treat operator, PostgreSQL image, storage and major-version changes as controlled production changes.